BUIDL CTC 2026 FALL · ATTESTCOIN PROTOCOL · AI TRACK

CreditPassport

Credit history you can prove, not just claim. Payments that provably happened on another chain become a credit profile on Creditcoin, underwritten by an agent whose discretion is bounded on-chain. No oracle operator anywhere.

Verified history

Every record is an `InvoicePaid` log decoded from a receipt whose inclusion Creditcoin's verifier precompile checked.

Bounded agent

The agent scores and explains. The contract computes the maximum credit limit from verified data and refuses anything above it.

Drawable credit

Payers draw cUSD against the limit on Creditcoin and repay. Limits rise only with more proven history.

PROBLEM

Credit needs history. History lives where payments happen.

Moving a payment record from one chain to a lender on another requires either a centralized oracle that asserts "this address paid its invoices," or self-reported data. Both put a trusted party between the facts and the lender, and both are how credit data gets faked.

The same gap shows up everywhere people ask for portable reputation:

> **"LinkedIn but for landlords and tenants"** — two-way verified rental history, r/SomebodyMakeThis

> **"Checks if the software you paid for actually works"** — pre-payment milestone proof for freelancers, r/SideProject

> **"Referral agreements with automated commission escrow"** — verbal promises lose money, r/SomebodyMakeThis

> **"A CFA-like credential that verifies real code"** — Ask HN

Demand signals collected Aug 2026 from Reddit and Hacker News

INSIGHT

Attestcoin turns a source-chain receipt into something a Creditcoin contract can check itself.

- Creditcoin attests Sepolia blocks continuously. On testnet the attestation trails the head by ~36 blocks, about 7 minutes.
- The hosted prover returns Merkle inclusion + continuity proofs and the transaction bytes, including the receipt.
- The verifier precompile at `0xFD2` checks the proof on-chain. `EvmV1Decoder` decodes the receipt's logs.
- So a contract can enforce: "this payer paid this payee this amount by this block" without anyone asserting it.

What that unlocks

A credit profile assembled only from proven payments, an underwriting agent that can read it but not inflate it, and a credit line whose ceiling is a pure function of verified data. The trust roots are Creditcoin's attestation and the owner's registration of which source contracts count. Nothing else.

HOW A PAYMENT BECOMES CREDIT

Five steps, two chains, zero oracles

1 Pay

Payer settles an invoice through `PaymentRail` on Sepolia. The log carries amount, due block and paid block.

2 Attest

Creditcoin attests the Sepolia block a few minutes later.

3 Prove

The agent fetches inclusion and continuity proofs, singly or ten at a time sharing one continuity proof.

4 Verify

`CreditPassport` asks the precompile to verify, decodes the receipt, keeps only rail logs, records on-time or late.

5 Underwrite

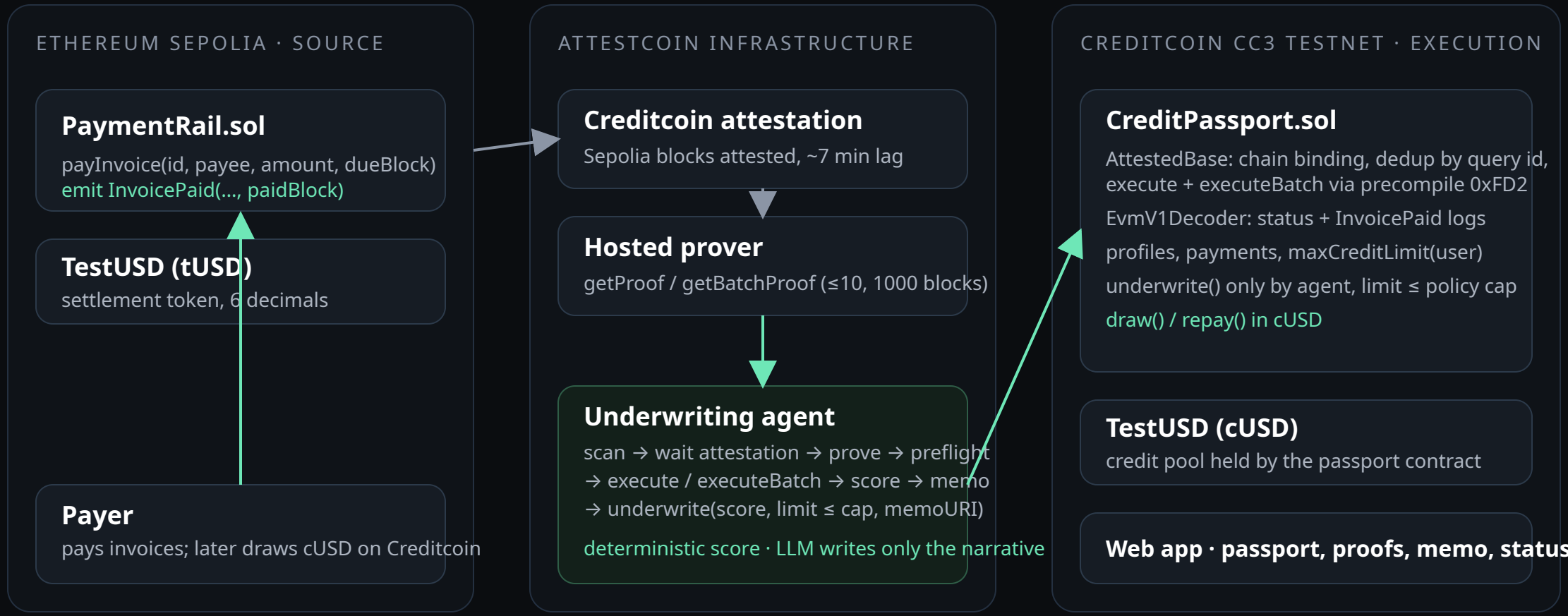
The agent scores the verified history, writes a memo, sets a limit the contract caps. The payer draws cUSD.

Lateness is decided by `dueBlock` and `paidBlock` fixed in the source log at payment time, never by whoever submits the proof.

Sequence diagram in README.md

ARCHITECTURE

Two chains, one prover, one agent



ATTESTCOIN PROTOCOL UTILISATION

Every layer of the protocol is exercised, and tested against real prover output

WHERE	ATTESTCOIN SURFACE	WHY IT MATTERS
`AttestedBase.execute`	INativeQueryVerifier.verifyAndEmit (single), calculateTxIndex	Inclusion + continuity verified on-chain; query id derivation identical to ASCBase for replay protection
`AttestedBase.executeBatch`	verifyAndEmit batch overload, shared continuity proof	Up to 10 payments imported in one transaction; not in the tutorials
`CreditPassport._processAndEmitEvent`	EvmV1Decoder.decodeReceiptFields, getLogsByEventSignature	Receipt status checked; only logs from the registered rail count
`agent/src/proofs.ts`	@gluwa/usc-sdk ProofBuilder.getProof / getBatchProof, waitUntilHeightAttested	Waits for attestation, fetches proofs, flattens batches into contract arguments
`agent/src/proofs.ts`	PrecompileBlockProver.verifySingle, PrecompileChainInfoProvider	Off-chain dry run before spending gas; live attestation lag in the UI
`test/RealProverBytes.t.sol`	Captured prover txBytes for a live Sepolia transaction	Decoder reproduces the Sepolia receipt field-for-field; execute() rejects it only for lacking a rail log
`checks/LivePrecompileCheck.sol`	Deploy + setSources + execute inside one	Real Sepolia ERC-20 transfers recorded (single

npm run cli -- verify · fetches a proof for any Sepolia tx and asks the precompile, no key, no gas

An agent you do not have to trust

The agent reads only what the contract verified, computes a deterministic score (0–1000) from punctuality, depth, tenure, volume and recency, and writes an underwriting memo. When a model key is configured, a language model writes the two-sentence narrative from the computed facts. It never produces a number.

Then it calls `underwrite(user, score, limit, memoURI)`. The contract computes the ceiling itself and reverts above it.

```
datedCap = 50% × datedVolume × onTime /
(onTime + late)
undatedCap = 25% × undatedVolume
if late > onTime then datedCap = 0
maxCreditLimit = datedCap + undatedCap

require(limit ≤ maxCreditLimit && limit ≥
drawn)
```

Demo: Alice, 6 proven invoices, 5 on time → score 814, cap 606.25 cUSD, agent extends 606. Bob, 1 on time, 2 late → score 477, cap 0, agent extends 0.

SECURITY MODEL

What an attacker would have to do

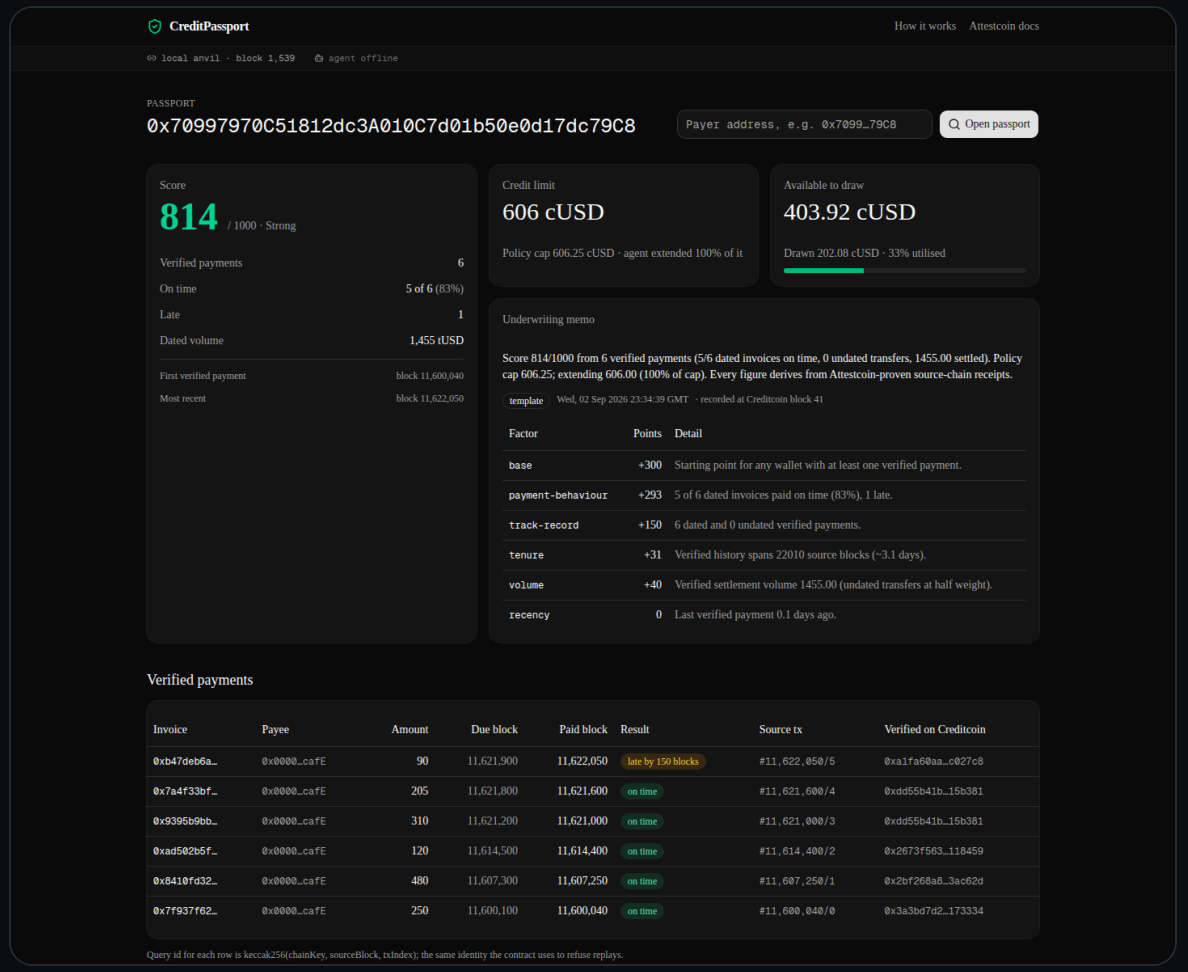
- **Fake a payment log.** Logs are accepted only from the registered rail; a look-alike contract emitting `InvoicePaid` is ignored, and a transaction with no rail log reverts.
- **Replay a proof.** Query id (chainKey, block, txIndex) is marked before the hook runs; batches skip processed entries and revert only if nothing is new.
- **Use another chain.** Proofs must carry the chain key fixed at deployment.
- **Prove a failed transaction.** Receipt status is checked; reverted source transactions record nothing.
- **Backdate a payment.** Due and paid blocks come from the source log fixed at payment time.
- **Bribe the agent.** The agent's maximum is a pure function of verified history. A compromised agent can under-extend, never over-extend, and cannot lower a limit below what is drawn.

Trust roots: Creditcoin's attestation of Sepolia, and the owner's registration of the rail and token addresses. There is no oracle operator to compromise.

Every claim above has a Foundry test in contracts/test

DEMO

A passport, end to end



Score, credit limit and available credit at the top; the agent's memo with its factor table; every verified payment with the source block and its Creditcoin verification transaction; the on-chain cap breakdown; and the underwriting history.

Two rows share one Creditcoin transaction: they were imported together through `executeBatch`.

```
$ npm run cli -- verify
tx 0x197cff58…68ec0
block 11622748 (latest attested 11622750)
proof fetched in 82 ms: 8 Merkle siblings, 3 continuity roots
verifier precompile 0xFD2 on Creditcoin says VALID
```

Local demo: scripts/demo-local.sh · Testnet: see README for addresses

STATUS

What exists today

Contracts

AttestedBase, CreditPassport, PaymentRail, TestUSD. 32 tests including genuine prover bytes. Deploy scripts for both chains.

Agent

Single and batch proof submission with off-chain preflight, deterministic scoring, memos as data: URIs, status API, CLI for pay / prove / underwrite / profile / verify.

Web

Passport dashboard with live attestation lag and agent queue. Empty, loading and error states.

Deployed on CC3 testnet

`CreditPassport: see README · PaymentRail (Sepolia): see README`

Roadmap

- Ethereum mainnet as source (chain key 3) and more rails: ERC-3009 transfers, payroll, subscriptions
- Creditcoin-native lenders consuming passports as collateral signals
- Attestcoin write-ability to push approved lines back to the source chain
- Scanned-statement ingestion via the same verify-not-trust pattern

github: creditpassport/ in the submission repository